1. **Script to run number of times**

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

c=1
while [ $c -le 5 ]
do
        echo "Welcone $c times"
        (( c++ ))
done
```

```
[ec2-user@ip-172-31-45-159 ~]$ sudo su -
Last login: Fri Oct 16 04:16:08 UTC 2026 on pts/1
[root@ip-172-31-45-159 ~]# ls
ayesha  saam
[root@ip-172-31-45-159 ~]# cd saam
[root@ip-172-31-45-159 saam]# vi do-while.bash
[root@ip-172-31-45-159 saam]# chmod 755 do-while.bash
[root@ip-172-31-45-159 saam]# ./do-while.bash
Welcone 1 times
Welcone 2 times
Welcone 3 times
Welcone 4 times
Welcone 5 times
[root@ip-172-31-45-159 saam]#
```

Steps for this While Loop

1. Start the Bash script.
2. Initialize the variable c with the value 1.
   This means the counting starts from 1.
3. Check the condition:
   Is c less than or equal to 5?
4. If the condition is TRUE, execute the statements inside the loop.
5. Display the message showing the current value of c.
   For example: Welcome 1 times.
6. Increase the value of c by 1.
   So, 1 becomes 2, then 3, and so on.
7. Check the condition again.
   The loop continues as long as c is less than or equal to 5.
8. When c becomes 6, the condition becomes FALSE.
9. The loop stops, and the script finishes.

2. Script to run for a number of seconds

Steps for this Script

1. Start the Bash script.
2. Initialize count as 0.
   This variable keeps track of how many times the loop has executed.
3. Initialize num as 10.
   This represents the number of seconds remaining.
4. Check the while-loop condition:
   Is count less than 10?
5. If the condition is TRUE, enter the loop.
6. Display the number of seconds remaining along with the process name provided as an argument.
7. Wait for 1 second.
   This creates the countdown effect.
8. Decrease the value of num by 1.
   For example: 10 → 9 → 8 → 7 → ... → 1.
9. Increase the value of count by 1.
   For example: 0 → 1 → 2 → ... → 10.
10. Check the while condition again.
    The loop continues until count reaches 10.
11. When count becomes 10, the condition becomes FALSE, so the loop stops.
12. Display the final message saying that the specified process has been stopped.

```
[root@ip-172-31-45-159 saam]# vi do-while1.bash
[root@ip-172-31-45-159 saam]# chmod 755 do-while1.bash
[root@ip-172-31-45-159 saam]# ./do-while1.bash

10 seconds left to stop this process


9 seconds left to stop this process


8 seconds left to stop this process


7 seconds left to stop this process


6 seconds left to stop this process


5 seconds left to stop this process


4 seconds left to stop this process


3 seconds left to stop this process


2 seconds left to stop this process


1 seconds left to stop this process


process is stopped!!!

[root@ip-172-31-45-159 saam]#
```

3. case Scripts:

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

echo
echo Please chose one of the options below
echo
echo 'a = Display Date and Time'
echo 'b = List file and directories'
echo 'c = List users logged in'
echo 'd = Check System uptime'
echo

        read choices

        case $choices in

a) date;;
b) ls;;
c) who;;
d) uptime;;
*) echo Invalid choice - Bye.

                esac
```

```
[root@ip-172-31-45-159 saam]# vi case-scripts.bash
[root@ip-172-31-45-159 saam]# chmod case-scripts.bash
chmod: missing operand after ‘case-scripts.bash’
Try 'chmod --help' for more information.
[root@ip-172-31-45-159 saam]# chmod 755 case-scripts.bash
[root@ip-172-31-45-159 saam]# ./case-scripts.bash

Please chose one of the options below

a = Display Date and Time
b = List file and directories
c = List users logged in
d = Check System uptime

a
Fri Oct 16 08:59:30 UTC 2026
[root@ip-172-31-45-159 saam]#
```

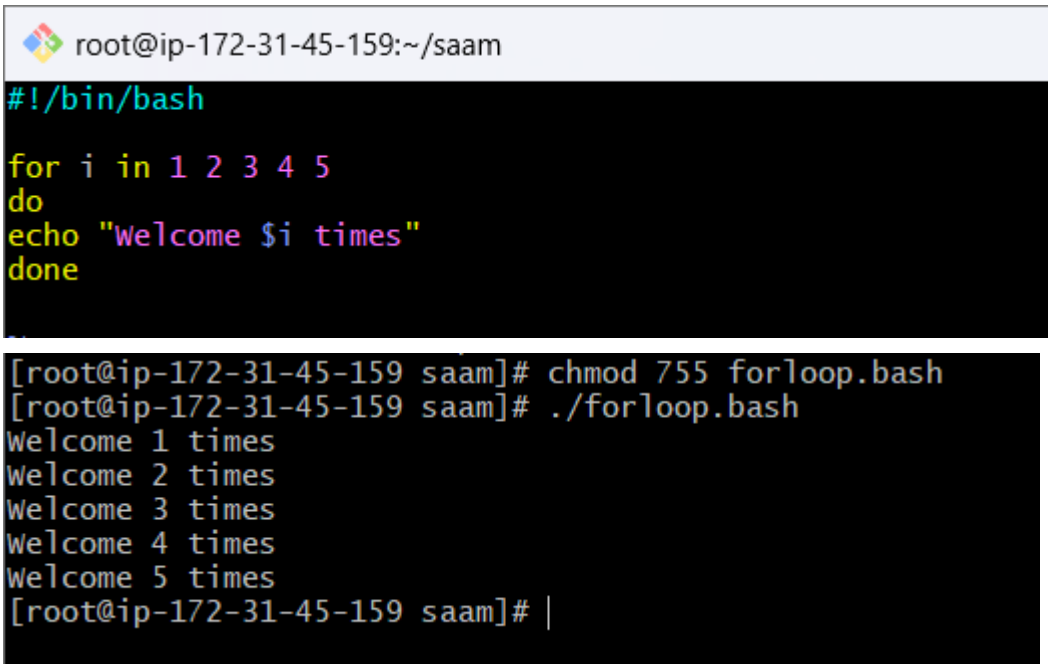
Steps for this CASE Statement

1. Start the Bash script.
2. Display a message asking the user to choose an option.
3. Display the four available options:

- a → Display date and time
- b → List files and directories
- c → Display users currently logged in
- d → Display system uptime

4. Take input from the user.
   The user's selected option is stored for checking.
5. Start the CASE statement to compare the user's choice with the available options.
6. If the user selects a, display the current date and time.
7. If the user selects b, display the files and directories in the current location.
8. If the user selects c, display the users who are currently logged into the system.
9. If the user selects d, display how long the system has been running.
10. If the user enters anything other than a, b, c, or d, display an "Invalid choice" message.
11. End the CASE statement and finish the script.

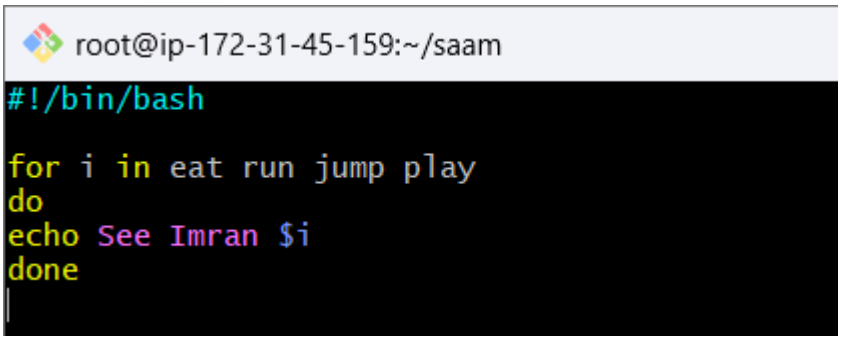
4. Simple for loop output



Steps for this FOR Loop

1. Start the Bash script.
2. Create a for loop with the values 1, 2, 3, 4, and 5.
3. The loop takes the first value, 1, and stores it in the loop variable.
4. Display the welcome message with the current value.
5. The loop moves to the next value, 2, and displays the message again.
6. The same process continues for 3, 4, and 5.
7. After processing the value 5, there are no more values left.
8. The loop ends and the script finishes.

5. Write a Bash script using a for loop to iterate through the values eat, run, jump, and play, and display a message for each value in the format: See Imran <value>.




```
[root@ip-172-31-45-159 saam]# vi forloop1.bash
[root@ip-172-31-45-159 saam]# chmod 755 forloop1.bash
[root@ip-172-31-45-159 saam]# ./forloop1.bash
See Imran eat
See Imran run
See Imran jump
See Imran play
```

Steps for this FOR Loop

1. Start the Bash script.
2. Create a for loop with four values:
    - eat
    - run
    - jump
    - play
3. The loop takes the first value, eat, and stores it in the loop variable.
4. Display the message using the current value: eat.
5. The loop takes the next value, run, and displays the message.
6. The same process continues for jump and play.
7. After processing all four values, there are no more values left.
8. The loop ends and the script finishes.

6. for loop to create 5 files named 1-5

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

for i in {1..5}
do
   touch $i
done
~
~
~
~
~
~
```

Steps for this FOR Loop

1. Start the Bash script.
2. Create a for loop that contains the numbers 1 to 5.
3. The loop takes the first number, 1, and stores it in the loop variable.
4. Create a file using the current number as the file name.
5. The loop moves to the next number, 2, and creates another file.
6. The same process continues for 3, 4, and 5.
7. After processing all five numbers, there are no more values left.
8. The loop ends and the script finishes.

7. If-then Scripts:

Check the variable

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

count=100
if [ $count -eq 100 ]
then
  echo Count is 100
else
  echo Count is not 100
fi

~
```

```
root@ip-172-31-45-159:~/saam
[root@ip-172-31-45-159 saam]# vi if-then.bash
[root@ip-172-31-45-159 saam]# chmod 755 if-then.bash
[root@ip-172-31-45-159 saam]# ./if-then.bash
Count is 100
[root@ip-172-31-45-159 saam]#
```

Steps for this IF-ELSE Script

1. Start the Bash script.
2. Assign the value 100 to the variable count.
3. Check the condition to see whether the value of count is equal to 100.
4. If the condition is TRUE, display "Count is 100".
5. If the condition is FALSE, execute the else part and display "Count is not 100".
6. End the IF-ELSE statement and finish the script.

8. Check if a file error.txt exist

```
root@ip-172-31-45-159:~/saam
#!/bin/bash

clear
if [ -e /home/iafzal/error.txt ]

        then
        echo "File exist"
        else
        echo "File does not exist"
fi
```

```
root@ip-172-31-45-159:~/saam
File does not exist
[root@ip-172-31-45-159 saam]#
```

Steps for this IF-ELSE File Checking Script

1. Start the Bash script.
2. Clear the terminal screen so that the previous output is removed.
3. Check whether the file /home/iafzal/error.txt exists.
4. If the file exists, display "File exist".
5. If the file does not exist, execute the else part and display "File does not exist".
6. End the IF-ELSE statement and finish the script.

9. Exit status

```
[root@ip-172-31-45-159 saam]# pwd
/root/saam
[root@ip-172-31-45-159 saam]# echo $?
0
[root@ip-172-31-45-159 saam]#
```

Start the Bash script.

Execute the pwd command.
This displays the current working directory.

The pwd command completes its execution.

Check the exit status of the previous command using echo $?.

If pwd executes successfully, its exit status is 0.

Therefore, the output will show:

- The current directory from pwd
- 0 from echo $?

1.1 Create a bash script to check if a directory is available or not.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

if [ -d /home/ayesha ]
then
    echo "Directory exists"
else
    echo "Directory does not exist"
fi
~
~
~
~
```

root@ip-172-31-45-159:~/saam

```
[root@ip-172-31-45-159 saam]# vi directory.bash
[root@ip-172-31-45-159 saam]# chmod 755 directory.bash
[root@ip-172-31-45-159 saam]# ./directory.bash
Directory does not exist
[root@ip-172-31-45-159 saam]#
```

Steps

1. Start the Bash script.
2. Check whether the specified path is a directory.
3. The -d condition checks whether the directory exists at the given location.
4. If the directory exists, the if condition becomes true.
5. Display "Directory exists".
6. If the directory does not exist, the condition becomes false.
7. Execute the else section and display "Directory does not exist".
8. End the IF-ELSE statement and finish the script.

1.2 Create a bash script to create multiple files.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

for i in 1 2 3 4 5
do
    touch "file$i.txt"
done
~
```

```
[root@ip-172-31-45-159 saam]# vi directory.bash
[root@ip-172-31-45-159 saam]# chmod 755 directory.bash
[root@ip-172-31-45-159 saam]# ./directory.bash
Directory does not exist
[root@ip-172-31-45-159 saam]# vi multiple-files.bash
[root@ip-172-31-45-159 saam]# chmod 755 multiple-files.bash
[root@ip-172-31-45-159 saam]# ./multiple-files.bash
[root@ip-172-31-45-159 saam]# ls multiple-files.bash
multiple-files.bash
[root@ip-172-31-45-159 saam]# ll
total 40
-rw-r--r--. 1 root root   0 Oct 16 09:10 1
-rw-r--r--. 1 root root   0 Oct 16 09:10 2
-rw-r--r--. 1 root root   0 Oct 16 09:10 3
-rw-r--r--. 1 root root   0 Oct 16 09:10 4
-rw-r--r--. 1 root root   0 Oct 16 09:10 5
-rwxr-xr-x. 1 root root 345 Oct 16 08:58 case-scripts.bash
-rwxr-xr-x. 1 root root 113 Oct 16 09:24 directory.bash
-rwxr-xr-x. 1 root root  95 Oct 16 08:51 do-while.bash
-rwxr-xr-x. 1 root root 242 Oct 16 08:54 do-while1.bash
-rw-r--r--. 1 root root   0 Oct 16 09:27 file1.txt
-rw-r--r--. 1 root root   0 Oct 16 09:27 file2.txt
-rw-r--r--. 1 root root   0 Oct 16 09:27 file3.txt
-rw-r--r--. 1 root root   0 Oct 16 09:27 file4.txt
-rw-r--r--. 1 root root   0 Oct 16 09:27 file5.txt
-rwxr-xr-x. 1 root root  65 Oct 16 09:04 forloop.bash
-rwxr-xr-x. 1 root root  67 Oct 16 09:07 forloop1.bash
-rwxr-xr-x. 1 root root  50 Oct 16 09:10 forloop2.bash
-rwxr-xr-x. 1 root root 103 Oct 16 09:17 if-then.bash
-rwxr-xr-x. 1 root root 144 Oct 16 09:18 if-then1.bash
-rwxr-xr-x. 1 root root  63 Oct 16 09:26 multiple-files.bash
drwxr-xr-x. 2 root root   6 Oct 16 04:17 scripts
```

Steps

1. Start the Bash script.
2. Create a for loop with the values 1, 2, 3, 4, and 5.
3. The loop takes the first value, 1.
4. Create a file using that value as part of the filename.
5. The loop moves to the next value, 2, and creates the next file.
6. The same process continues for 3, 4, and 5.
7. After all five values are processed, the loop ends.
8. Five files are created in the current directory.

1.3 Create a bash script to take a backup of a directory.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash
source="/home/ayesha/scripts"
backup="/home/ayesha/scripts_backup.tar.gz"
tar -czf "$backup" "$source"
echo "Backup completed succesfully"
~
```

```
[root@ip-172-31-45-159 ~]# vi backup.bash
[root@ip-172-31-45-159 ~]# chmod 755 backup.bash
[root@ip-172-31-45-159 ~]# ./backup.bash
Backup completed successfully
[root@ip-172-31-45-159 ~]#
```

Steps

1. Start the Bash script.
2. Specify the directory that needs to be backed up.
3. Specify the location and name for the backup file.
4. Create a compressed backup of the specified directory.
5. The contents of the directory are collected into a single backup file.
6. The backup is compressed to reduce its size.
7. The backup file is saved at the specified backup location.
8. Display a message confirming that the backup was completed successfully.

1.4 Create a bash script to install Nginx on an EC2 server.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

echo "Updating packages..."
sudo apt update -y

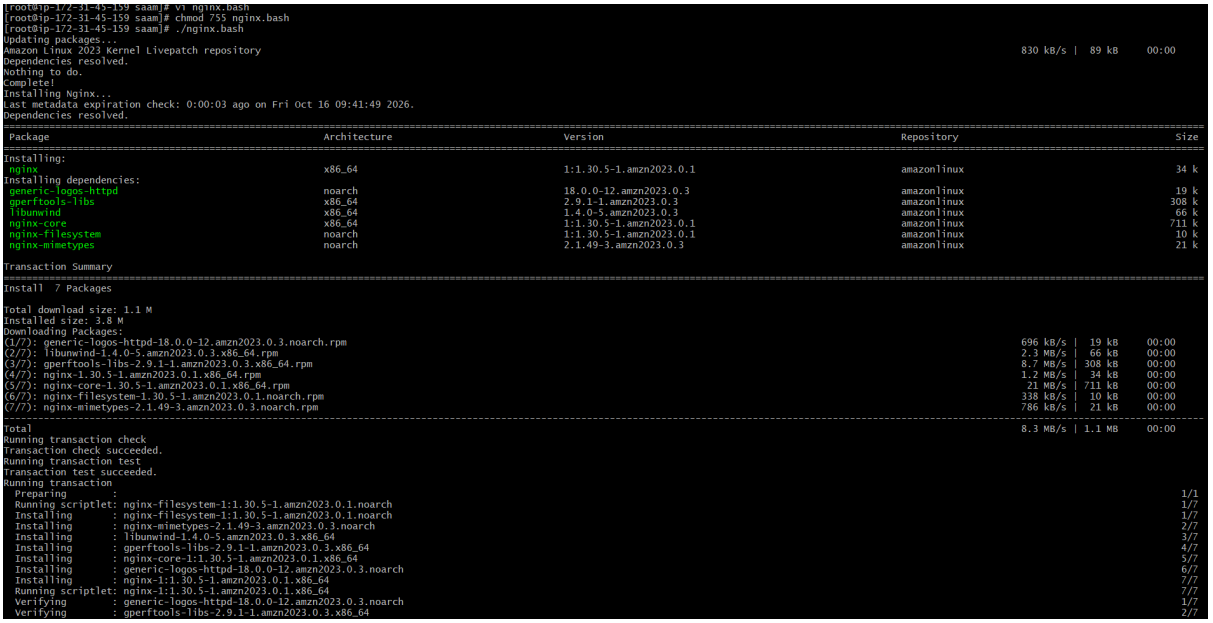
echo "Installing Nginx..."
sudo apt install nginx -y

echo "Starting Nginx..."
sudo systemctl start nginx

echo "Enabling Nginx..."
sudo systemctl enable nginx

echo "Checking Nginx status..."
sudo systemctl status nginx --no-pager

echo "Nginx installation completed successfully"
~
```



Steps

1. Start the Bash script.
2. Update the package information on the EC2 server so the system has the latest package information.
3. Install Nginx using the system's package manager.
4. Start the Nginx service after installation.
5. Enable Nginx so that it automatically starts when the EC2 server is rebooted.
6. Check the Nginx service status to confirm whether it is running successfully.
7. Display a completion message after the installation process.

1.5 Create a bash script to install Apache Tomcat on an EC2 server

root@ip-172-31-45-159:~/saam

```
#!/bin/bash


echo "Updating packages..."
sudo yum update -y

echo "Installing Java..."
sudo yum install openjdk-17-jdk -y

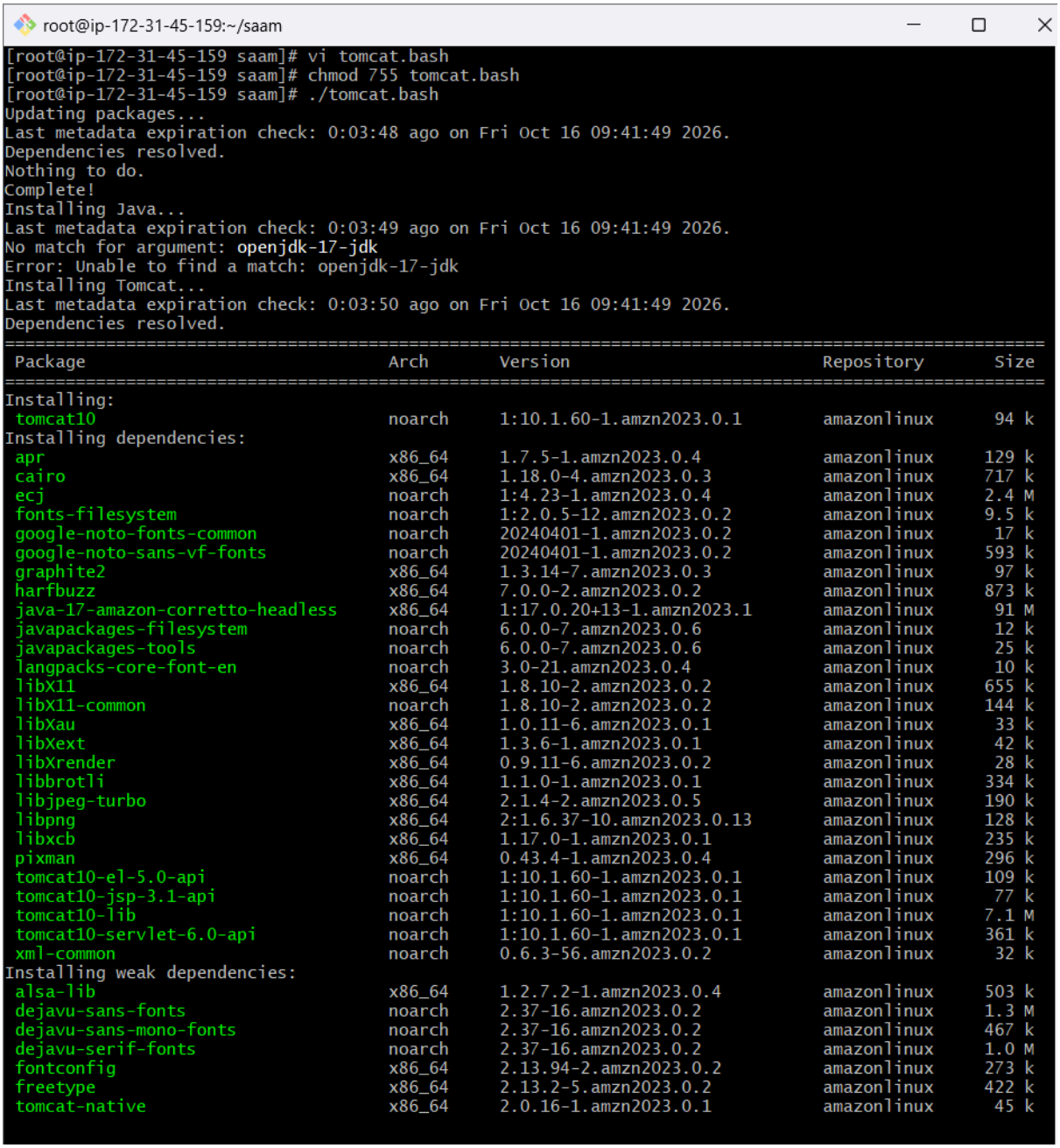
echo "Installing Tomcat..."
sudo yum install tomcat10 -y

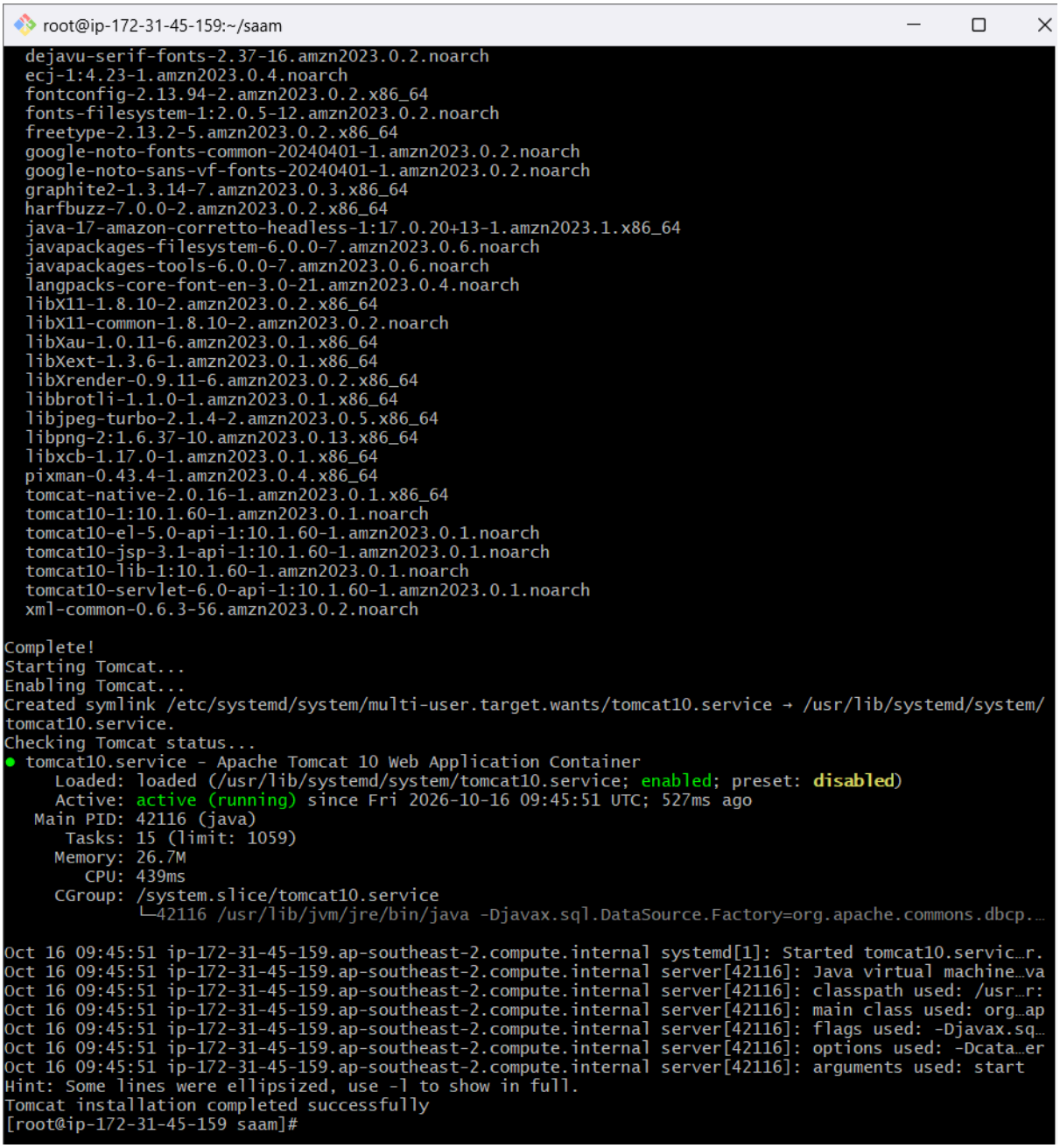
echo "Starting Tomcat..."
sudo systemctl start tomcat10

echo "Enabling Tomcat..."
sudo systemctl enable tomcat10

echo "Checking Tomcat status..."
sudo systemctl status tomcat10 --no-pager

echo "Tomcat installation completed successfully"
~
~
```


root@ip-172-31-45-159:~/saam
[root@ip-172-31-45-159 saam]# vi tomcat.bash
[root@ip-172-31-45-159 saam]# chmod 755 tomcat.bash
[root@ip-172-31-45-159 saam]# ./tomcat.bash
Updating packages...
Last metadata expiration check: 0:03:48 ago on Fri Oct 16 09:41:49 2026.
Dependencies resolved.
Nothing to do.
Complete!
Installing Java...
Last metadata expiration check: 0:03:49 ago on Fri Oct 16 09:41:49 2026.
No match for argument: openjdk-17-jdk
Error: Unable to find a match: openjdk-17-jdk
Installing Tomcat...
Last metadata expiration check: 0:03:50 ago on Fri Oct 16 09:41:49 2026.
Dependencies resolved.
Package Arch Version Repository Size
Installing:
tomcat10 noarch 1:10.1.60-1.amzn2023.0.1 amazonlinux 94 k
Installing dependencies:
apr x86_64 1.7.5-1.amzn2023.0.4 amazonlinux 129 k
cairo x86_64 1.18.0-4.amzn2023.0.3 amazonlinux 717 k
ecj noarch 1:4.23-1.amzn2023.0.4 amazonlinux 2.4 M
fonts-filesystem noarch 1:2.0.5-12.amzn2023.0.2 amazonlinux 9.5 k
google-noto-fonts-common noarch 20240401-1.amzn2023.0.2 amazonlinux 17 k
google-noto-sans-vf-fonts noarch 20240401-1.amzn2023.0.2 amazonlinux 593 k
graphite2 x86_64 1.3.14-7.amzn2023.0.3 amazonlinux 97 k
harfbuzz x86_64 7.0.0-2.amzn2023.0.2 amazonlinux 873 k
java-17-amazon-corretto-headless x86_64 1:17.0.20+13-1.amzn2023.1 amazonlinux 91 M
javapackages-filesystem noarch 6.0.0-7.amzn2023.0.6 amazonlinux 12 k
javapackages-tools noarch 6.0.0-7.amzn2023.0.6 amazonlinux 25 k
langpacks-core-font-en noarch 3.0-21.amzn2023.0.4 amazonlinux 10 k
libX11 x86_64 1.8.10-2.amzn2023.0.2 amazonlinux 655 k
libX11-common noarch 1.8.10-2.amzn2023.0.2 amazonlinux 144 k
libXau x86_64 1.0.11-6.amzn2023.0.1 amazonlinux 33 k
libXext x86_64 1.3.6-1.amzn2023.0.1 amazonlinux 42 k
libXrender x86_64 0.9.11-6.amzn2023.0.2 amazonlinux 28 k
libbrotli x86_64 1.1.0-1.amzn2023.0.1 amazonlinux 334 k
libjpeg-turbo x86_64 2.1.4-2.amzn2023.0.5 amazonlinux 190 k
libpng x86_64 2:1.6.37-10.amzn2023.0.13 amazonlinux 128 k
libxcb x86_64 1.17.0-1.amzn2023.0.1 amazonlinux 235 k
pixman x86_64 0.43.4-1.amzn2023.0.4 amazonlinux 296 k
tomcat10-el-5.0-api noarch 1:10.1.60-1.amzn2023.0.1 amazonlinux 109 k
tomcat10-jsp-3.1-api noarch 1:10.1.60-1.amzn2023.0.1 amazonlinux 77 k
tomcat10-lib noarch 1:10.1.60-1.amzn2023.0.1 amazonlinux 7.1 M
tomcat10-servlet-6.0-api noarch 1:10.1.60-1.amzn2023.0.1 amazonlinux 361 k
xml-common noarch 0.6.3-56.amzn2023.0.2 amazonlinux 32 k
Installing weak dependencies:
alsa-lib x86_64 1.2.7.2-1.amzn2023.0.4 amazonlinux 503 k
dejavu-sans-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.3 M
dejavu-sans-mono-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 467 k
dejavu-serif-fonts noarch 2.37-16.amzn2023.0.2 amazonlinux 1.0 M
fontconfig x86_64 2.13.94-2.amzn2023.0.2 amazonlinux 273 k
freetype x86_64 2.13.2-5.amzn2023.0.2 amazonlinux 422 k
tomcat-native x86_64 2.0.16-1.amzn2023.0.1 amazonlinux 45 k



Steps

1. Start the Bash script.
2. Update the package information on the EC2 server.
3. Install Java, because Tomcat requires Java to run.
4. Install Apache Tomcat using the package manager.
5. Start the Tomcat service after installation.
6. Enable Tomcat so that it automatically starts when the EC2 server is rebooted.
7. Check the Tomcat service status to verify that Tomcat is running.
8. Display a completion message after the installation is finished.

1.6 Create a bash script to check if the Nginx service is running, if not running then script should start the service.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

if systemctl is-active --quiet nginx
then
    echo "Nginx is already running"
else
    echo "Nginx is not running. Starting Nginx..."
    sudo systemctl start nginx
    echo "Nginx started successfully"
fi
~
~
~
```

root@ip-172-31-45-159:~/saam

```
[root@ip-172-31-45-159 saam]# vi nginx-status.bash
[root@ip-172-31-45-159 saam]# chmod 755 nginx-status.bash
[root@ip-172-31-45-159 saam]# ./nginx-status.bash
Nginx is already running
[root@ip-172-31-45-159 saam]#
```

Steps

1. Start the Bash script.
2. Check the current status of the Nginx service.
3. If Nginx is already running, display "Nginx is already running".
4. If Nginx is not running, start the Nginx service.
5. Display a message confirming that Nginx has been started.
6. End the script.

1.7 Create a bash script for a calculator.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

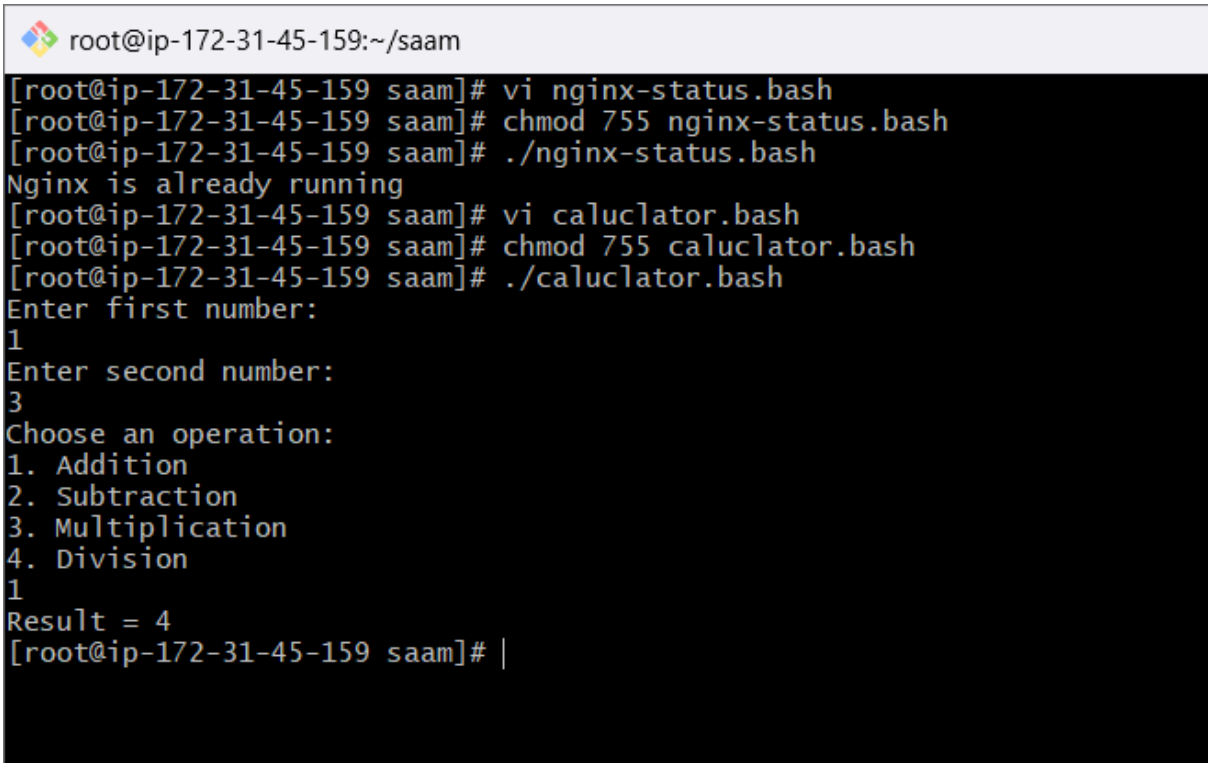
echo "Enter first number:"
read num1

echo "Enter second number:"
read num2

echo "Choose an operation:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

read choice

case $choice in
    1)
        result=$((num1 + num2))
        echo "Result = $result"
        ;;
    2)
        result=$((num1 - num2))
        echo "Result = $result"
        ;;
    3)
        result=$((num1 * num2))
        echo "Result = $result"
        ;;
    4)
        result=$((num1 / num2))
        echo "Result = $result"
        ;;
    *)
        echo "Invalid choice"
        ;;
esac
```



Steps

1. Start the Bash script.
2. Ask the user to enter the first number.
3. Ask the user to enter the second number.
4. Display the calculator options:
    - Addition
    - Subtraction
    - Multiplication
    - Division
5. Read the user's choice.
6. Use a case statement to identify the selected operation.
7. Perform the calculation.
8. Display the result.
9. If the user enters an invalid option, display "Invalid choice".
10. End the script.

1.8 Create a bash script to check if a directory exists, if not then create a directory.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

dir="/home/ayesha/mydata"

if [ -d "$dir" ]
then
    echo "Directory already exists"
else
    echo "Directory does not exist. Creating directory..."
    mkdir -p "$dir"
    echo "Directory created successfully"
fi
~
```

root@ip-172-31-45-159:~/saam

```
[root@ip-172-31-45-159 saam]# vi check-directory.bash
[root@ip-172-31-45-159 saam]# chmod 755 check-directory.bash
[root@ip-172-31-45-159 saam]# ./check-directory.bash
Directory does not exist. Creating directory...
Directory created successfully
[root@ip-172-31-45-159 saam]#
```

Steps

1. Start the Bash script.
2. Define the directory path that needs to be checked.
3. Check whether the directory exists using the directory condition.
4. If the directory exists, display "Directory already exists".
5. If the directory does not exist, create the directory.
6. Display "Directory created successfully".
7. End the script.

1.9 Create a bash script to delete the last 3 lines of a file.

root@ip-172-31-45-159:~/saam

```
#!/bin/bash

file="/home/ayesha/test.txt"

head -n -3 "$file" > temp.txt
mv temp.txt "$file"

echo "Last 3 lines deleted successfully"
~
~
~
~
```

```
[root@ip-172-31-45-159 ayesha]# vi delete.bash
[root@ip-172-31-45-159 ayesha]# chmod 755 delete.bash
[root@ip-172-31-45-159 ayesha]# ./delete.bash
Last 3 lines deleted successfully
[root@ip-172-31-45-159 ayesha]#
```

Steps

1. Start the Bash script.
2. Specify the file whose last 3 lines need to be deleted.
3. Read the file and keep all lines except the last 3 lines.
4. Store the remaining content in a temporary file.
5. Replace the original file with the temporary file.
6. Display a confirmation message.
7. End the script.

1.10 Bash script to monitor cpu and if it is more than 80% then send email notification.

root@ip-172-31-45-159:~/saam/scripts

```
#!/bin/bash

CPU=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')

THRESHOLD=80
EMAIL="your-email@example.com"

if (( ${CPU%.*} > THRESHOLD ))
then
    echo "CPU usage is ${CPU}% on $(hostname)" | mail -s "High CPU Usage Alert" "$EMAIL"
    echo "CPU usage is ${CPU}%. Email notification sent."
else
    echo "CPU usage is ${CPU}%. CPU usage is normal."
fi
~
~
```

root@ip-172-31-45-159:~/saam/scripts

```
[root@ip-172-31-45-159 saam]# cd scripts
[root@ip-172-31-45-159 scripts]# vi cpu-monitor.bash
[root@ip-172-31-45-159 scripts]# chmod 755 cpu-monitor.bash
[root@ip-172-31-45-159 scripts]# ./cpu-monitor.bash
CPU usage is 3.2%. CPU usage is normal.
[root@ip-172-31-45-159 scripts]#
```

Steps

1. Start the Bash script.
2. Get the current CPU usage from the system.
3. Store the CPU usage in the CPU variable.
4. Set the threshold to 80%.
5. Compare the current CPU usage with 80%.
6. If CPU usage is more than 80%, send an email notification.
7. If CPU usage is 80% or below, display that CPU usage is normal.
8. End the script.

1.11 Bash script to monitor disk space and if it is more than 80% then send email notification.

root@ip-172-31-45-159:~/saam/scripts

```
#!/bin/bash

DISK=$(df / | awk 'NR==2 {print $5}' | tr -d '%')

THRESHOLD=80
EMAIL="ayesha.erml@gmail.com"

if [ "$DISK" -gt "$THRESHOLD" ]
then
    echo "Disk usage is ${DISK}% on $(hostname)" | mail -s "High Disk Usage Alert" "$EMAIL"
    echo "Disk usage is ${DISK}%. Email notification sent."
else
    echo "Disk usage is ${DISK}%. Disk usage is normal."
fi
~
~
~
```

root@ip-172-31-45-159:~/saam/scripts

```
[root@ip-172-31-45-159 scripts]# vi disk-space.bash
[root@ip-172-31-45-159 scripts]# chmod 755 disk-space.bash
[root@ip-172-31-45-159 scripts]# ./disk-space.bash
Disk usage is 25%. Disk usage is normal.
[root@ip-172-31-45-159 scripts]#
```

Steps

1. Start the Bash script.
2. Check the disk usage of the / (root) filesystem.
3. Store the current disk usage percentage in the DISK variable.
4. Set the threshold to 80%.
5. Compare the disk usage with 80%.
6. If disk usage is more than 80%, send an email notification.
7. If disk usage is 80% or below, display that disk usage is normal.
8. End the script.

1.12 Bash script to monitor memory and if it is more than 80% then send email notification

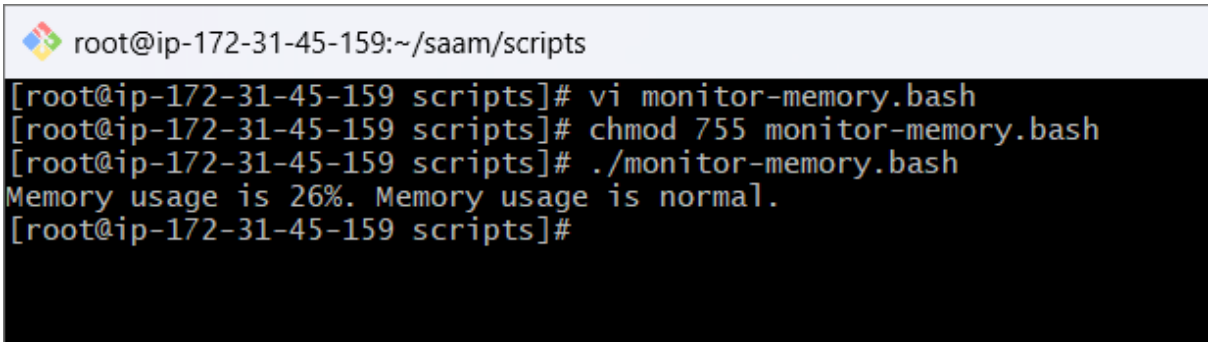
root@ip-172-31-45-159:~/saam/scripts

```
#!/bin/bash

MEMORY=$(free | awk '/Mem:/ {printf "%.0f", ($3/$2)*100}')

THRESHOLD=80
EMAIL="your-email@example.com"

if [ "$MEMORY" -gt "$THRESHOLD" ]
then
    echo "Memory usage is ${MEMORY}% on $(hostname)" | mail -s "High Memory Usage Alert" "$EMAIL"
    echo "Memory usage is ${MEMORY}%. Email notification sent."
else
    echo "Memory usage is ${MEMORY}%. Memory usage is normal."
fi
~
```



Steps

1. Start the Bash script.
2. Check the current memory usage of the system.
3. Calculate the memory usage as a percentage.
4. Store the percentage in the MEMORY variable.
5. Set the threshold to 80%.
6. Compare the current memory usage with 80%.
7. If memory usage is more than 80%, send an email notification.
8. If memory usage is 80% or below, display that memory usage is normal.
9. End the script.

Exercise: Crontab Entries

2.1 . April 5th Midnight

```
root@ip-172-31-45-159:~/saam/scripts
#!/bin/bash

echo "Cron job executed on $(date)" >> /home/ayesha/cron_output.txt
~
```

```
root@ip-172-31-45-159:~/saam/scripts
Installing dependencies:
 cronie-anacron          x86_64          1.5.7-1.amzn2023.0.2          amazonlinux          32 k

Transaction Summary
================================================================================
Install  2 Packages

Total download size: 147 k
Installed size: 341 k
Downloading Packages:
(1/2): cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64.rpm                1.2 MB/s |  32 kB     00:00
(2/2): cronie-1.5.7-1.amzn2023.0.2.x86_64.rpm                        3.2 MB/s | 115 kB     00:00
--------------------------------------------------------------------------------
Total                                                                1.6 MB/s | 147 kB     00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                             1/1
  Installing       : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64                  1/2
  Running scriptlet: cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64                  1/2
  Installing       : cronie-1.5.7-1.amzn2023.0.2.x86_64                          2/2
  Running scriptlet: cronie-1.5.7-1.amzn2023.0.2.x86_64                          2/2
Created symlink /etc/systemd/system/multi-user.target.wants/crond.service → /usr/lib/systemd/system/cro
nd.service.

  Verifying        : cronie-1.5.7-1.amzn2023.0.2.x86_64                          1/2
  Verifying        : cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64                  2/2

Installed:
  cronie-1.5.7-1.amzn2023.0.2.x86_64            cronie-anacron-1.5.7-1.amzn2023.0.2.x86_64

Complete!
[root@ip-172-31-45-159 scripts]# sudo systemctl start crond
[root@ip-172-31-45-159 scripts]# sudo systemctl enable crond
[root@ip-172-31-45-159 scripts]# sudo systemctl status crond
● crond.service - Command Scheduler
     Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-16 10:26:26 UTC; 17s ago
   Main PID: 43874 (crond)
      Tasks: 1 (limit: 1059)
     Memory: 996.0K
        CPU: 9ms
     CGroup: /system.slice/crond.service
             └─43874 /usr/sbin/crond -n

Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal systemd[1]: Started crond.service - C>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) STARTUP (1.5.7)
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (Syslog wil>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (RANDOM_DEL>
Oct 16 10:26:26 ip-172-31-45-159.ap-southeast-2.compute.internal crond[43874]: (CRON) INFO (running wi>
[root@ip-172-31-45-159 scripts]# crontab -e
no crontab for root - using an empty one
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
[root@ip-172-31-45-159 scripts]#
```

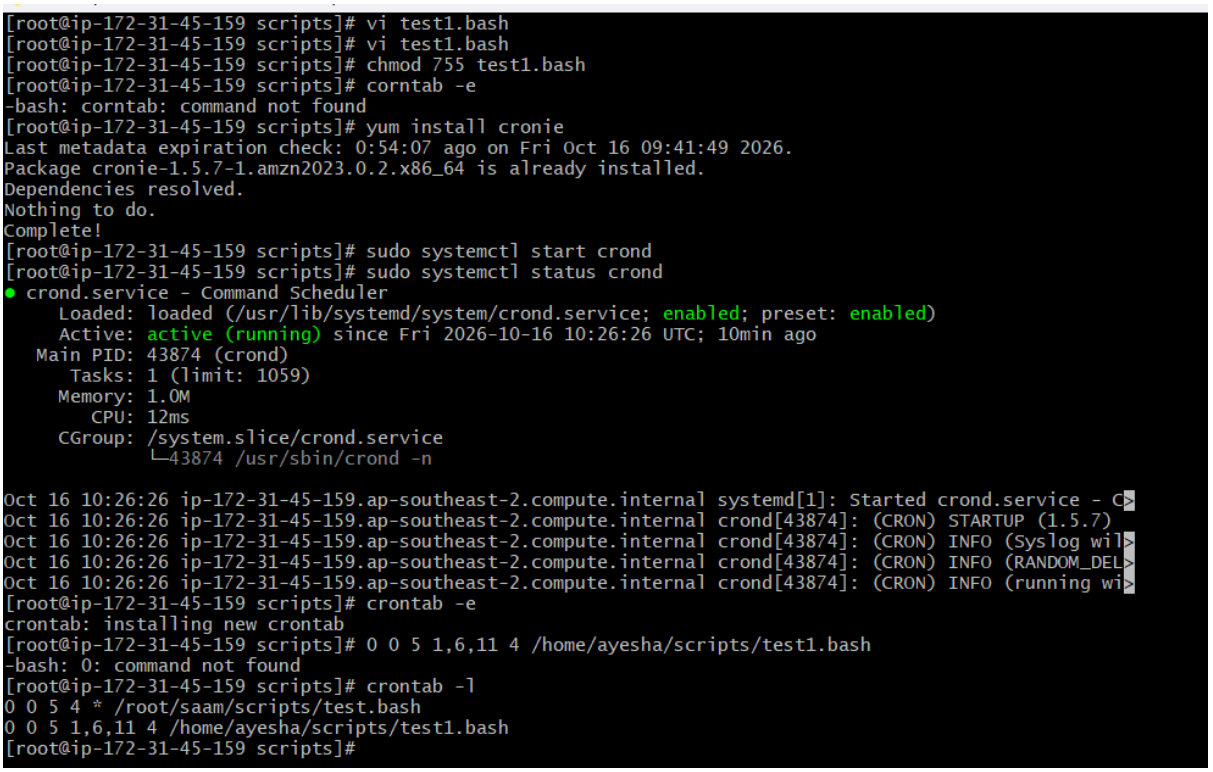
Steps — April 5th at Midnight

1. I checked the operating system of my EC2 instance.
2. I checked whether the crontab command was available.
3. I installed the required cron package if crontab was not available.
4. I started the cron service.
5. I enabled the cron service so that it starts automatically after a reboot.
6. I created a Bash script in my scripts directory.
7. I gave execute permission to my Bash script.
8. I opened the crontab for editing.

9. I added the schedule for April 5th at midnight.
10. I specified the absolute path of my Bash script in the cron entry.
11. I saved and exited the crontab.
12. I verified the cron entry using crontab -l.
13. I configured the cron job to execute my script every year on April 5th at 12:00 AM.

2.2 5th of Every November, January, June if it is a Thursday

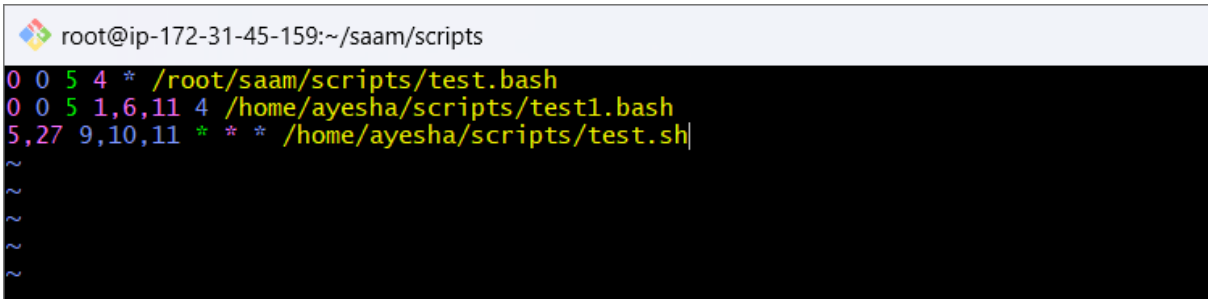


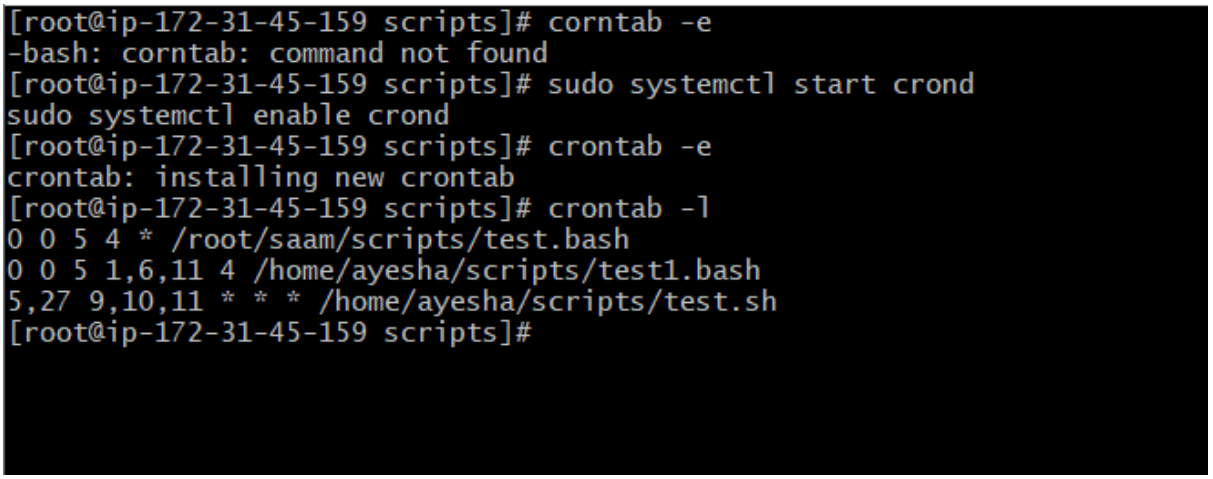


1. I created my Bash script in the scripts directory.
2. I gave execute permission to the script.
3. I opened my crontab using crontab -e.

4. I added the cron schedule for the 5th of January, June, and November on Thursday.
5. I saved and exited the crontab.
6. I verified my cron entry using crontab -l.
7. I temporarily changed the schedule to every minute to test the script.
8. I checked the output to confirm that the cron job executed successfully.
9. I restored the original schedule after testing.

2.3 At 05 and 27th minutes of 9, 10, 11 hours every day



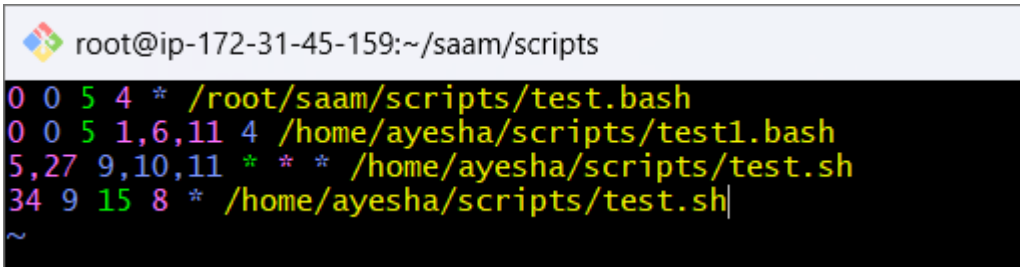


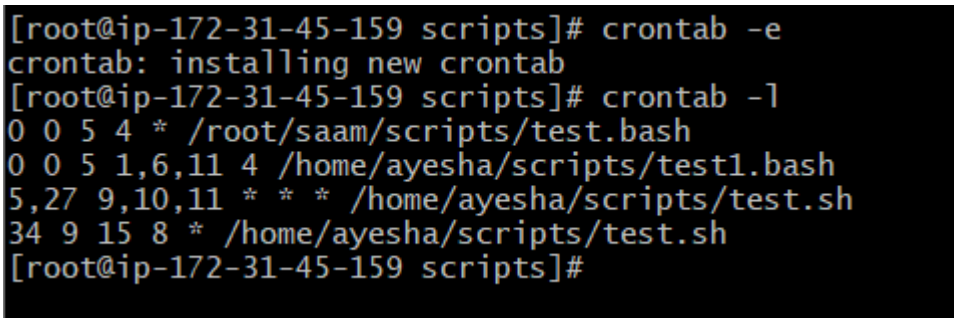
Steps — 05th and 27th Minutes of 9, 10, 11 Hours Every Day

1. I created my Bash script test.sh in the scripts directory.
2. I added the command to record the cron execution date and time.
3. I gave execute permission to my test.sh script.
4. I started the crond service once on my EC2 instance.
5. I enabled the crond service to start automatically after a reboot.
6. I opened my crontab using crontab -e.
7. I added the cron schedule for the 5th and 27th minutes of the 9th, 10th, and 11th hours.
8. I saved and exited the crontab.
9. I verified my cron entry using crontab -l.
10. I used the same test.sh script instead of creating a new script.

11. I configured the cron job to run every day at 09:05, 09:27, 10:05, 10:27, 11:05, and 11:27.

2.4 34 minutes of 9th hour on 15th August

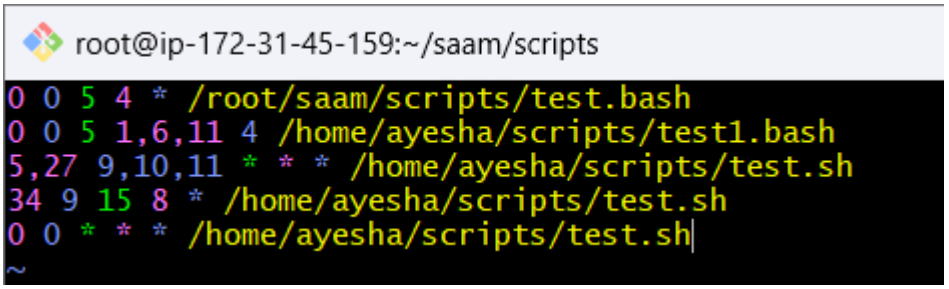




Steps — 34 Minutes of 9th Hour on 15th August

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the schedule for the 34th minute of the 9th hour on 15th August.
4. I saved and exited the crontab.
5. I verified the cron entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every year on 15th August at 9:34 AM.

2.5 Every midnight



```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

Steps — Every Midnight

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the schedule to run the script at midnight every day.
4. I saved and exited the crontab.
5. I verified the cron entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every day at 12:00 AM (midnight).

2.6 Every Weekend (Saturday night 11:59 PM)

```
root@ip-172-31-45-159:~/saam/scripts
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
~
```

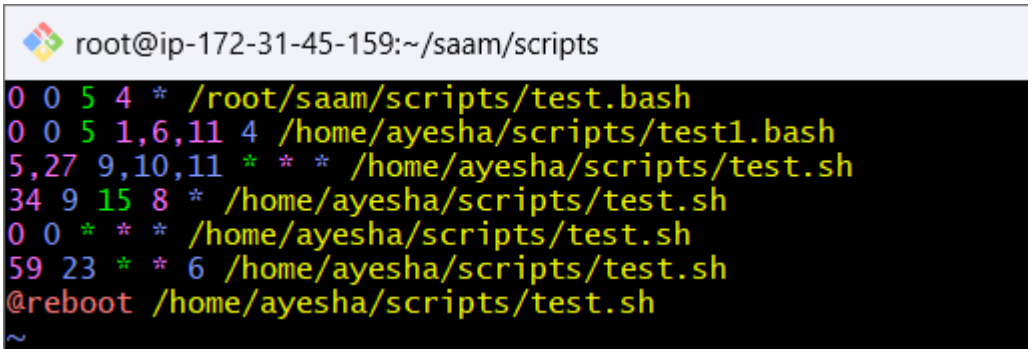
```
[root@ip-172-31-45-159 scripts]# crontab -e
crontab: installing new crontab
[root@ip-172-31-45-159 scripts]# crontab -l
0 0 5 4 * /root/saam/scripts/test.bash
0 0 5 1,6,11 4 /home/ayesha/scripts/test1.bash
5,27 9,10,11 * * * /home/ayesha/scripts/test.sh
34 9 15 8 * /home/ayesha/scripts/test.sh
0 0 * * * /home/ayesha/scripts/test.sh
59 23 * * 6 /home/ayesha/scripts/test.sh
[root@ip-172-31-45-159 scripts]#
```

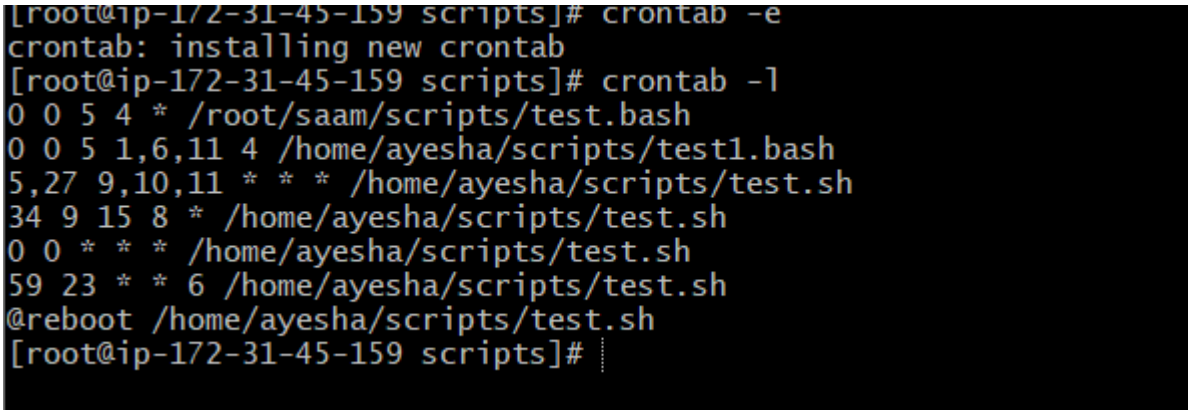
Steps

1. I used my existing Bash script test.sh.

2. I opened the crontab using crontab -e.
3. I added the schedule to run the script every Saturday at 11:59 PM.
4. I saved and exited the crontab.
5. I verified the entry using crontab -l.
6. I kept the crond service running without restarting it.
7. I configured the cron job to execute my script every Saturday night at 11:59 PM.

2.7 After every reboot





Steps

1. I used my existing Bash script test.sh.
2. I opened the crontab using crontab -e.
3. I added the @reboot entry to execute my script whenever the system reboots.
4. I saved and exited the crontab.
5. I verified the entry using crontab -l.
6. I configured the cron job to run my script automatically after every system reboot.